

VoiceCredit Tracker

Voice-Based Credit Tracking System for Informal Market
Traders

Version: v1.0.0

Status: Development

Last Updated: September 2026

Documentation for the VoiceCredit Tracker Project

Project Lead: Eleanor Sanyu Nabaloga

Email: support@voicecredit.ug

Repository: <https://github.com/voicecredit/tracker>

Confidential - For Internal Use Only

Contents

List of Figures	2
List of Tables	3
1 Introduction	4
1.1 Purpose	4
1.2 Target Audience	4
2 Problem Statement	4
2.1 The Challenge	4
2.2 Solution Approach	5
3 Target Users	5
3.1 User Persona: Mama Grace	5
3.2 Pain Points	5
3.3 Daily Routine	5
3.4 User Statistics (Uganda Context)	6
4 Core Features	6
4.1 Feature Overview	6
4.2 User Journey	6
5 Technology Stack	7
5.1 Backend (Offline-First)	7
5.2 Frontend (Mobile-First)	7
5.3 Infrastructure	7
6 Installation & Setup	8
6.1 Prerequisites	8
6.2 Local Development Setup	8
6.2.1 Backend Setup	8
6.2.2 Mobile App Setup (React Native)	8
6.2.3 Mobile App Setup (Flutter)	9
6.3 Environment Variables	9
7 Usage Guide	10
7.1 Getting Started (User Perspective)	10
7.1.1 Step 1: Download & Install	10
7.1.2 Step 2: First Launch	10
7.1.3 Step 3: Basic Commands	10
7.1.4 Step 4: Voice Feedback	10
7.2 Advanced Features	10
7.2.1 Bulk Recording	10
7.2.2 Modify a Record	11
7.2.3 Daily Report	11
7.3 Offline Operation	11
8 Voice Interface Design	11

8.1	Speech Recognition Engine	11
8.1.1	Offline Engine: Vosk	11
8.1.2	Cloud Engine: Whisper (Optional)	11
8.2	NLP Intent Recognition	12
8.3	Voice Feedback System	12
9	Architecture	13
9.1	System Architecture Diagram	13
9.2	Data Flow	13
9.3	Database Schema	13
9.4	Design Patterns	14
10	Offline Functionality	14
10.1	Offline Architecture	14
10.2	Offline Capabilities	15
10.3	Sync Strategy	16
11	Security & Privacy	16
11.1	Privacy by Design	16
11.1.1	No Tracking of Tax-Identifiable Data	16
11.2	Data Encryption	16
11.3	Permission Model	17
11.4	Data Retention Policy	17
12	Testing	17
12.1	Test Coverage	17
12.2	Running Tests	18
12.3	Test Scenarios	18
12.3.1	Voice Recognition Accuracy	18
12.3.2	Offline Mode Tests	18
13	Deployment	19
13.1	Mobile App Deployment	19
13.1.1	Google Play Store	19
13.1.2	Apple App Store	19
13.2	Backend Deployment	19
13.2.1	Docker Container	19
13.2.2	Cloud Deployment	20
13.3	Monitoring	20
14	Contributing	20
14.1	Contribution Guidelines	20
14.2	Coding Standards	21
14.2.1	Python (Backend)	21
14.2.2	JavaScript/TypeScript (Frontend)	21
14.2.3	Testing Requirements	21
14.3	Language Contributions	21
15	License	22

16 Support & Contact	22
17 Changelog	23
17.1 [v1.0.0] - 2026-09-15	23
17.2 [v0.9.0] - 2026-08-20	23
18 Roadmap	23
18.1 Q4 2026	23
18.2 Q1 2027	24
18.3 Q2 2027	24
19 Success Stories	24
20 Troubleshooting	24
20.1 Common Issues	24
20.2 Error Codes	25
A Voice Command Reference	25
A.1 Luganda Commands	25
A.2 Swahili Commands	25
B Offline Model Specifications	26
C Cost Estimates	26
C.1 Development Costs	26
C.2 Scalability Costs (10,000 Users)	26

List of Figures

1	User Journey Flow	6
2	System Architecture Diagram	13

List of Tables

1	Challenges Faced by Informal Market Traders	4
2	Target User Statistics (Uganda)	6
3	Core Features Overview	6
4	Backend Technology Stack	7
5	Frontend Technology Stack	7
6	Infrastructure Stack	7
7	Basic Voice Commands	10
8	Design Patterns	14
9	Offline vs Online Capabilities	15
10	Permission Model	17
11	Test Coverage Targets	17
12	Support Resources	23
13	Common Issues and Solutions	25
14	Error Codes	25
15	Luganda Voice Commands	25
16	Swahili Voice Commands	25
17	Offline Model Specifications	26
18	Development Costs (Monthly)	26
19	Scalability Costs (10,000 Users)	26

1 Introduction

1.1 Purpose

VoiceCredit Tracker is a voice-first mobile application designed specifically for informal market traders in Uganda and similar environments. It enables users to:

- **Record credit transactions** using natural voice in local languages
- **Track debts** without requiring reading or writing skills
- **Generate voice summaries** of daily sales and outstanding payments
- **Operate offline** in areas with poor internet connectivity
- **Maintain privacy** by not requiring tax identification or government registration

The system uses speech-to-text technology adapted for local languages (Luganda, Swahili, English) and provides audio feedback to confirm all actions.

1.2 Target Audience

This documentation is intended for:

- **Developers:** Understanding the architecture and contributing to the codebase
- **Project Managers:** Overseeing the development and deployment
- **End Users:** Understanding how to use the application
- **Stakeholders:** Evaluating the project's impact and potential

2 Problem Statement

2.1 The Challenge

Informal market traders like Mama Grace face multiple challenges, as illustrated in Table 1.

Table 1: Challenges Faced by Informal Market Traders

Challenge	Impact
Paper records lost/damaged	Rain, theft, and wear cause data loss
Low financial literacy	Only 10-15% can read balance sheets
Text-based digital alienation	50-60% struggle with text-heavy apps
Tax authority fears	Avoid digital tools that might reveal income
Unreliable internet	Market areas have poor connectivity

2.2 Solution Approach

VoiceCredit Tracker addresses these challenges by:

- **Eliminating text interfaces** - Everything is voice-based
- **Working offline** - Syncs when connection returns
- **Requiring no tax numbers** - No government identification needed
- **Using local languages** - Natural voice interaction in familiar languages
- **Providing audio summaries** - Listen to your business performance

3 Target Users

3.1 User Persona: Mama Grace

User Persona: Mama Grace

Demographics:

- Female market trader, 45+ years
- Sells vegetables/food items in open-air market
- Limited formal education (primary level)
- Speaks local language (Luganda/Swahili) as primary

Literacy Profile:

- **General Literacy:** Can read basic sentences (70-75% of informal workers)
- **Financial Literacy:** Minimal (10-15%)
- **Digital Literacy:** Low, intimidated by text-heavy apps

3.2 Pain Points

- ✗ Loses paper records to rain/damage
- ✗ Cannot use accounting software
- ✗ Anxious about digital tax tracking
- ✗ Forgetful of who owes money
- ✗ Unreliable market internet

3.3 Daily Routine

06:00 AM - Arrive at market, set up stall
07:00 AM - Open for business, serve customers
12:00 PM - Lunch break
05:00 PM - Close stall, reconcile accounts

3.4 User Statistics (Uganda Context)

Table 2 shows key statistics for the target user base.

Table 2: Target User Statistics (Uganda)

Metric	Percentage	Number
Informal workers	-	7.35 million
Basic literacy	70-75%	5.14-5.51 million
Financial literacy	10-15%	735K-1.1 million
Digital alienation	50-60%	3.7-4.4 million

4 Core Features

4.1 Feature Overview

Table 3 provides an overview of key features.

Table 3: Core Features Overview

Feature	Description	Voice Command Example
Record Credit	Log a customer debt	"Naguze matooke tatu, credit ya John"
Record Payment	Mark a debt as paid	"John yasasula matooke tatu"
Daily Summary	Get voice playback of transactions	"Mbulira ebya leero"
Customer Lookup	Check a customer's balance	"John alina nyinji?"
Debt Reminder	Get list of outstanding debts	"Ani alina bize?"
Report Generation	Generate voice report	"Nkwata raporo"

4.2 User Journey

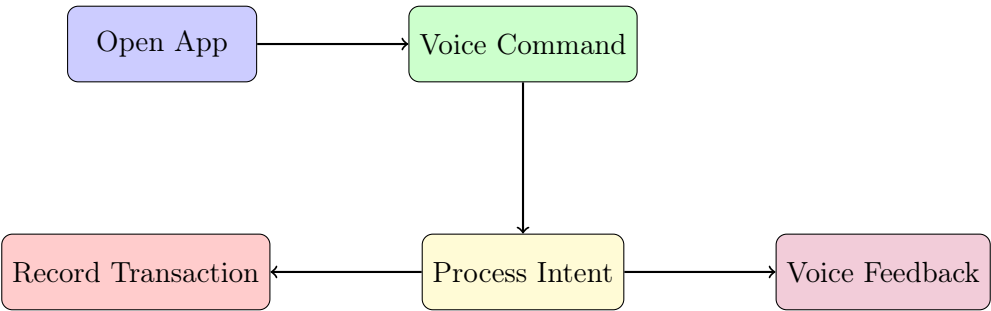


Figure 1: User Journey Flow

5 Technology Stack

5.1 Backend (Offline-First)

Table 4 lists the backend technologies.

Table 4: Backend Technology Stack

Component	Technology	Purpose
Runtime	Node.js / Python	Application server
Framework	Express / FastAPI	API endpoints
Database	SQLite / PouchDB	Offline-first local storage
Sync DB	CouchDB / MongoDB Atlas	Cloud sync when online
Speech-to-Text	Vosk API / Whisper	Offline speech recognition
Text-to-Speech	espeak / Google TTS	Voice feedback
NLP	spaCy / Custom parser	Intent recognition

5.2 Frontend (Mobile-First)

Table 5 lists the frontend technologies.

Table 5: Frontend Technology Stack

Component	Technology	Purpose
Framework	React Native / Flutter	Cross-platform mobile app
Voice Recorder	react-native-audio	Record voice input
Local Storage	AsyncStorage / SQLite	Offline data persistence
State Management	Redux / Riverpod	App state management

5.3 Infrastructure

Table 6 lists infrastructure components.

Table 6: Infrastructure Stack

Component	Technology	Purpose
Hosting	DigitalOcean / AWS	Cloud deployment
File Storage	S3-compatible storage	Audio file storage
CI/CD	GitHub Actions	Automated deployment
Monitoring	Sentry / LogRocket	Error tracking

6 Installation & Setup

6.1 Prerequisites

- Node.js v18+ or Python 3.9+
- npm/yarn or pip
- Mobile device or emulator (Android/iOS)
- Microphone access permission

6.2 Local Development Setup

6.2.1 Backend Setup

```
1 # Clone repository
2 git clone https://github.com/your-org/voicecredit-tracker.git
3 cd voicecredit-tracker/backend
4
5 # Install dependencies
6 pip install -r requirements.txt
7 # or
8 npm install
9
10 # Set up environment variables
11 cp .env.example .env
12 # Edit .env with your configuration
13
14 # Initialize database
15 python scripts/init_db.py
16
17 # Start development server
18 python app.py
19 # or
20 npm run dev
```

6.2.2 Mobile App Setup (React Native)

```
1 cd mobile
2
3 # Install dependencies
4 npm install
5
6 # iOS specific (Mac only)
7 cd ios && pod install && cd ..
8
9 # Start Metro bundler
10 npm start
```

```
11
12 # Run on Android
13 npm run android
14
15 # Run on iOS
16 npm run ios
```

6.2.3 Mobile App Setup (Flutter)

```
1 cd mobile_flutter
2
3 # Install dependencies
4 flutter pub get
5
6 # Run on Android
7 flutter run
8
9 # Run on iOS
10 flutter run -d ios
```

6.3 Environment Variables

```
1 # Backend
2 PORT=3000
3 NODE_ENV=development
4 DATABASE_URL=sqlite:///./voicecredit.db
5 OFFLINE_MODE=true
6 MAX_AUDIO_DURATION=30
7
8 # Speech Recognition
9 STT_ENGINE=vosk # Options: vosk, whisper, google
10 VOSK_MODEL_PATH=./models/vosk-model-luganda-0.1
11 WHISPER_MODEL=tiny
12
13 # Sync
14 CLOUD_SYNC_ENABLED=false
15 SYNC_INTERVAL=3600 # seconds
16 CLOUD_DB_URL=mongodb://localhost:27017/voicecredit
17
18 # Security
19 ENCRYPTION_KEY=your-secret-key
20 AUDIT_LOG_ENABLED=false # No tax tracking!
21
22 # Languages
23 SUPPORTED_LANGUAGES=["luganda", "swahili", "english"]
24 DEFAULT_LANGUAGE=luganda
```

7 Usage Guide

7.1 Getting Started (User Perspective)

7.1.1 Step 1: Download & Install

- Download from Google Play Store / App Store
- Grant microphone permissions when prompted

7.1.2 Step 2: First Launch

Welcome! Tap the microphone button to start.
Speak your language: [Luganda, Swahili, English]

7.1.3 Step 3: Basic Commands

Table 7 shows basic voice commands.

Table 7: Basic Voice Commands

Action	Voice Command (Luganda)	Voice Command (English)
Record credit	"Naguze [item] [qty], credit ya [name]"	"Give [customer] [qty] of [item] on credit"
Record payment	"[customer] yajasula [amount]"	"[customer] paid [amount]"
Daily summary	"Mbulira ebya leero"	"Today's summary"
Check balance	"[customer] alina nyinji?"	"How much does [customer] owe?"
All debts	"Ani alina bize?"	"Who owes me?"

7.1.4 Step 4: Voice Feedback

You: "Naguze matooke tatu, credit ya John"
Phone: "Recorded. John owes three bunches of bananas."

You: "John yajasula matooke tatu"
Phone: "Payment received. John paid for three bunches of bananas."

7.2 Advanced Features

7.2.1 Bulk Recording

You: "Naguze matooke tatu credit ya John, naguze obutama bubiri credit ya Mary"
Phone: "Recorded. John owes three bunches of bananas. Mary owes two bunches of matooke."

7.2.2 *Modify a Record*

You: "Sazawo John credit ya matooke tatu"

Phone: "Removed John's debt of three bunches of bananas."

7.2.3 *Daily Report*

You: "Mbulira ebya leero"

Phone: "Today's sales: 45,000 UGX. Credit: 25,000 UGX. Cash: 20,000 UGX.

Outstanding debts: John 15,000 UGX, Mary 10,000 UGX.

Total owed: 125,000 UGX."

7.3 Offline Operation

The app works 100% offline:

- **All data saved locally** on the device
- **Voice recognition** works without internet
- **Queue system** for syncing when network returns
- **No data sent** to cloud unless user chooses to sync

8 Voice Interface Design

8.1 Speech Recognition Engine

8.1.1 *Offline Engine: Vosk*

- **Model size:** ~40MB (lightweight)
- **Languages:** Luganda, Swahili, English
- **Accuracy:** 85-90% in controlled environments
- **Latency:** <500ms on modern smartphones

8.1.2 *Cloud Engine: Whisper (Optional)*

- **Model size:** Tiny (75MB) - Large (3GB)
- **Languages:** 100+ languages supported
- **Accuracy:** 95%+ with cloud processing
- **Latency:** 2-5 seconds (requires internet)

8.2 NLP Intent Recognition

```
1 # Intent recognition examples
2 intents = {
3     'record_credit': [
4         'naguze (.) credit ya (.)', # Luganda
5         'give (.) (.) on credit',   # English
6         'mpe (.) kwa deni'          # Swahili
7     ],
8     'record_payment': [
9         '(.) yasasula (.)',          # Luganda
10        '(.) paid (.)',              # English
11        '(.) alilipa (.)'            # Swahili
12    ],
13    'get_summary': [
14        'mbulira ebya leero',        # Luganda
15        'today\'s summary',          # English
16        'muhtasari wa leo'          # Swahili
17    ]
18 }
```

8.3 Voice Feedback System

```
1 // TTS configurations
2 const voiceFeedback = {
3     confirmation: {
4         luganda: "Ndabidde",
5         english: "Recorded",
6         swahili: "Imerekodiwa"
7     },
8     error: {
9         luganda: "Tontegereza, kozesa olulimi lwo",
10        english: "Please repeat, speak clearly",
11        swahili: "Tafadhali rudia, sema waziwazi"
12    },
13    summary: {
14        luganda: "Wano waliwo ebikukwatako",
15        english: "Here's your summary",
16        swahili: "Hapa kuna muhtasari wako"
17    }
18 };
```

9 Architecture

9.1 System Architecture Diagram

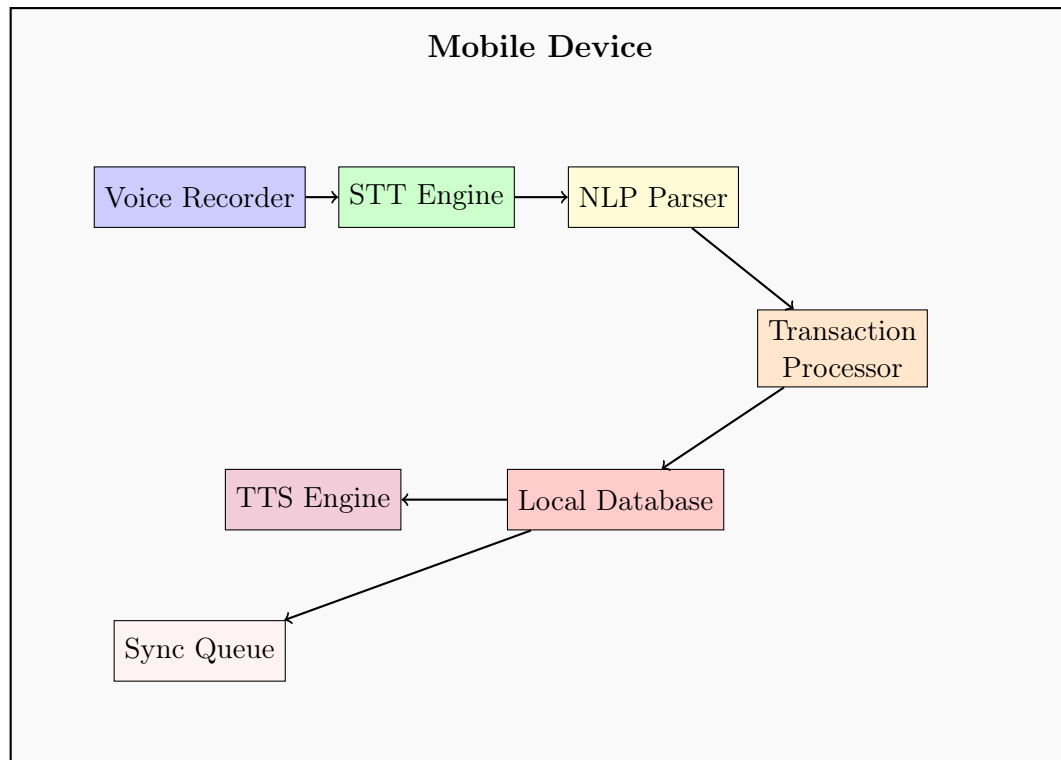


Figure 2: System Architecture Diagram

9.2 Data Flow

1. User speaks
2. Audio recorded
3. STT converts to text
4. NLP extracts intent and entities
5. Transaction stored in SQLite with encryption
6. Voice confirmation played back to user
7. (When online) Queue syncs to cloud

9.3 Database Schema

```
1 -- Local SQLite Schema
2 CREATE TABLE customers (
3     id TEXT PRIMARY KEY,
4     name TEXT,
5     phone TEXT,
```



```

6      created_at DATETIME
7  );
8
9  CREATE TABLE transactions (
10     id TEXT PRIMARY KEY,
11     type TEXT CHECK(type IN ('credit', 'payment', 'adjustment')),
12     customer_id TEXT,
13     item TEXT,
14     quantity INTEGER,
15     amount REAL,
16     description TEXT,
17     timestamp DATETIME,
18     synced BOOLEAN DEFAULT 0,
19     FOREIGN KEY (customer_id) REFERENCES customers(id)
20 );
21
22 CREATE TABLE audio_logs (
23     id TEXT PRIMARY KEY,
24     transaction_id TEXT,
25     audio_file_path TEXT,
26     transcript TEXT,
27     duration INTEGER,
28     timestamp DATETIME,
29     FOREIGN KEY (transaction_id) REFERENCES transactions(id)
30 );
31
32 -- No tax ID, no government identifiers in schema!

```

9.4 Design Patterns

Table 8 shows the design patterns used.

Table 8: Design Patterns

Pattern	Application	Benefit
Repository Pattern	Data access abstraction	Easy offline/online switching
Observer Pattern	Sync queue monitoring	Real-time sync status
Strategy Pattern	STT engine selection	Switch between offline/-cloud
Singleton Pattern	Database connection	Single point of access
Factory Pattern	Voice command creation	Flexible intent handling

10 Offline Functionality

10.1 Offline Architecture

```

1 class OfflineManager {
2   constructor() {
3     this.isOnline = navigator.onLine;
4     this.syncQueue = [];
5     this.localDB = new PouchDB('voicecredit');
6   }
7
8   async saveTransaction(transaction) {
9     // Always save locally first
10    const localId = await this.localDB.post(transaction);
11
12    // Queue for sync if online
13    if (this.isOnline) {
14      this.syncToCloud(transaction);
15    } else {
16      this.syncQueue.push(transaction);
17      this.showOfflineBanner();
18    }
19
20    return localId;
21  }
22
23  async syncQueue() {
24    if (!this.isOnline) return;
25
26    while (this.syncQueue.length > 0) {
27      const transaction = this.syncQueue.shift();
28      await this.syncToCloud(transaction);
29    }
30  }
31 }

```

10.2 Offline Capabilities

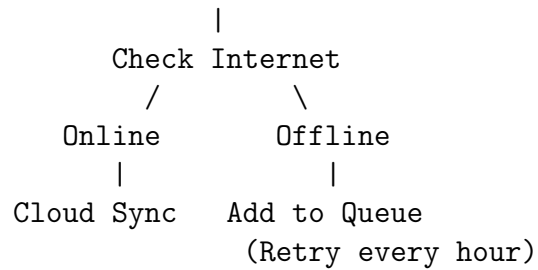
Table 9 shows offline vs online capabilities.

Table 9: Offline vs Online Capabilities

Feature	Offline	Online
Voice Recording	✓ Yes	✓ Yes
Speech-to-Text	✓ Yes (Vosk)	✓ Yes (Cloud)
Transaction Storage	✓ Yes	✓ Yes
Voice Feedback	✓ Yes	✓ Yes
Daily Summary	✓ Yes	✓ Yes
Cloud Backup	✗ No	✓ Yes
Multi-device Sync	✗ No	✓ Yes
Analytics	✗ No	✓ Yes

10.3 Sync Strategy

Immediate Save -> Local Storage



11 Security & Privacy

11.1 Privacy by Design

11.1.1 No Tracking of Tax-Identifiable Data

```
1 // Explicitly exclude tax-related fields
2 const privacyConfig = {
3   collectTaxID: false,
4   collectNationalID: false,
5   collectBusinessRegistration: false,
6   collectPIN: false,
7
8   // Only collect essential business data
9   collectCustomerName: true,
10  collectItemsSold: true,
11  collectAmounts: true,
12  collectDates: true
13 };
```

11.2 Data Encryption

```
1 const CryptoJS = require('crypto-js');
2
3 class EncryptionService {
4   encrypt(data) {
5     return CryptoJS.AES.encrypt(
6       JSON.stringify(data),
7       process.env.ENCRIPTION_KEY
8     ).toString();
9   }
10
11   decrypt(ciphertext) {
12     const bytes = CryptoJS.AES.decrypt(
13       ciphertext,
14       process.env.ENCRIPTION_KEY
```

```

15     );
16     return JSON.parse(bytes.toString(CryptoJS.enc.Utf8));
17 }
18 }

```

11.3 Permission Model

Table 10 shows the required permissions.

Table 10: Permission Model

Permission	Required	Reason
Microphone	✓ Yes	Voice input
Storage	✓ Yes	Save transactions
Internet	✗ No	Optional for sync
Contacts	✗ No	Privacy preserved
Location	✗ No	Not needed

11.4 Data Retention Policy

- Local data stored permanently until user deletes
- Cloud backup optional (user must opt-in)
- No data sharing with third parties
- No government reporting features
- User can delete all data at any time

12 Testing

12.1 Test Coverage

Table 11 shows test coverage targets.

Table 11: Test Coverage Targets

Test Type	Coverage Target	Tools
Unit Tests	80%+	Jest / PyTest
Integration Tests	70%+	Supertest
Voice Recognition	90%+ accuracy	Vosk testing suite
Offline Mode	100% coverage	Simulate network loss
UI/UX	All user flows	Detox / Appium

12.2 Running Tests

```
1 # Backend tests
2 cd backend
3 pytest tests/ -v --cov=app
4
5 # Frontend tests
6 cd mobile
7 npm test
8 npm run test:coverage
9
10 # E2E tests
11 npm run test:e2e
12
13 # Voice recognition tests
14 npm run test:voice
```

12.3 Test Scenarios

12.3.1 Voice Recognition Accuracy

```
1 describe('Voice Recognition', () => {
2   test('should recognize Luganda credit command', () => {
3     const audio = loadFixture('naguze_matooke_tatu.wav');
4     const result = sttEngine.transcribe(audio);
5     expect(result.text).toContain('naguze matooke tatu');
6     expect(result.confidence).toBeGreaterThan(0.8);
7   });
8
9   test('should handle background noise', () => {
10    const audio = loadFixture('noisy_market_recording.wav');
11    const result = sttEngine.transcribe(audio);
12    // Should still extract key transaction data
13    expect(result.transaction).toBeDefined();
14  });
15 });
```

12.3.2 Offline Mode Tests

```
1 describe('Offline Mode', () => {
2   test('should save transactions without internet', async () => {
3     // Simulate offline
4     mockNetworkConnection(false);
5
6     await recordTransaction({ customer: 'John', amount: 15000 });
7     const saved = await getLocalTransactions();
8
9     expect(saved).toHaveLength(1);
```

```
10     expect(saved[0].synced).toBe(false);
11   });
12
13   test('should sync when connection returns', async () => {
14     // Start offline, save transaction
15     mockNetworkConnection(false);
16     await recordTransaction({ customer: 'Mary', amount: 10000 });
17
18     // Restore connection
19     mockNetworkConnection(true);
20     await syncManager.processQueue();
21
22     const cloudData = await getCloudTransactions();
23     expect(cloudData).toContainEqual(
24       expect.objectContaining({ customer: 'Mary' })
25     );
26   });
27 });
```

13 Deployment

13.1 Mobile App Deployment

13.1.1 Google Play Store

```
1 # Generate signed APK
2 cd android
3 ./gradlew bundleRelease
4
5 # Upload to Play Store
6 # Open Google Play Console -> Create Release -> Upload AAB
```

13.1.2 Apple App Store

```
1 # Build for iOS
2 cd ios
3 pod install
4 # Open in Xcode -> Archive -> Upload to App Store
```

13.2 Backend Deployment

13.2.1 Docker Container

```
1 FROM python:3.9-slim
2
```

```
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install -r requirements.txt
7
8 COPY . .
9
10 ENV PORT=3000
11 ENV OFFLINE_MODE=true
12
13 EXPOSE 3000
14
15 CMD ["gunicorn", "--bind", "0.0.0.0:3000", "app:app"]
```

13.2.2 Cloud Deployment

```
1 # Build and push to Docker registry
2 docker build -t voicecredit-backend .
3 docker tag voicecredit-backend:latest registry.digitalocean.com/voicecredit/
   backend
4 docker push registry.digitalocean.com/voicecredit/backend
5
6 # Deploy to cloud
7 kubectl apply -f k8s/deployment.yaml
8 kubectl apply -f k8s/service.yaml
```

13.3 Monitoring

```
1 # Prometheus metrics configuration
2 metrics:
3   endpoints:
4     - /metrics
5   alerts:
6     - name: API Latency
7       condition: latency > 500ms
8     - name: Offline Users
9       condition: offline_users > 1000
10    - name: Sync Queue
11      condition: sync_queue > 5000
```

14 Contributing

14.1 Contribution Guidelines

1. Fork the repository

2. Create a feature branch

```
1 git checkout -b feature/voice-language-support
2
```

3. Commit with meaningful messages

```
1 git commit -m "feat: Add Swahili language support"
2
```

4. Push and create PR

```
1 git push origin feature/voice-language-support
2
```

14.2 Coding Standards

14.2.1 Python (Backend)

- Follow PEP 8
- Use type hints
- Write docstrings for all functions

14.2.2 JavaScript/TypeScript (Frontend)

- Follow Airbnb style guide
- Use ES6+ features
- Write JSDoc comments

14.2.3 Testing Requirements

- Unit tests for all new features
- Integration tests for API endpoints
- Voice recognition tests for new languages

14.3 Language Contributions

Contributing new language support:

```
1 # Add new language to config
2 LANGUAGES = {
3     'luganda': {
4         'model': 'vosk-model-luganda-0.2',
5         'intents': {...},
```



```
6         'tts_voice': 'luganda-voice'
7     },
8     'swahili': {
9         'model': 'vosk-model-swahili-0.1',
10        'intents': {...},
11        'tts_voice': 'swahili-voice'
12    },
13    # Add your language here
14 }
```

15 License

This project is licensed under the **MIT License** - see LICENSE file.

MIT License

Copyright (c) 2026 VoiceCredit Tracker

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

16 Support & Contact

Table [12](#) shows support resources.

Table 12: Support Resources

Resource	Link/Info
Project Lead	Eleanor Sanyu Nabaloga
Email	support@voicecredit.ug
Documentation	https://docs.voicecredit.ug
Issue Tracker	https://github.com/voicecredit/tracker/issues
Community Forum	https://community.voicecredit.ug
WhatsApp Support	+256-XXX-XXX-XXX

17 Changelog

17.1 [v1.0.0] - 2026-09-15

Added

- Voice-based credit recording in Luganda
- Offline-first architecture
- Voice confirmation system
- Daily summary reports

Fixed

- Initial sync issues
- Background noise handling

17.2 [v0.9.0] - 2026-08-20

Added

- Swahili language support
- Multi-language switching
- Customer history tracking

18 Roadmap

18.1 Q4 2026

- Add support for 3 more local languages
- Enhanced NLP for complex transactions
- Voice analytics dashboard

- SMS notifications for debt reminders

18.2 Q1 2027

- WhatsApp integration for business reports
- Group credit tracking (for cooperatives)
- AI-powered sales forecasting
- Inventory management voice commands

18.3 Q2 2027

- Cloud analytics for business insights
- Multi-stall/multi-location support
- Integration with mobile money (optional)
- Advanced privacy and encryption features

19 Success Stories

Success Story: Mama Grace

"I used to lose money every month because I couldn't remember who paid and who didn't. Now I just speak to my phone, and it remembers everything. Even my daughter helps me use it!"

Success Story: James

"I thought all digital tools would expose my business to government. But this one doesn't ask for my tax number. It just helps me track my money. I feel safe."

20 Troubleshooting

20.1 Common Issues

Table [13](#) provides solutions to common issues.

Table 13: Common Issues and Solutions

Issue	Solution
Microphone not working	Check permissions in phone settings
Voice not recognized	Speak clearly, reduce background noise
App crashes	Update to latest version, restart phone
Transactions not saving	Check storage space on device
Sync not working	Ensure network connection, retry

20.2 Error Codes

Table 14 shows error codes and solutions.

Table 14: Error Codes

Code	Description	Action
E001	Microphone unavailable	Grant microphone permission
E002	Storage full	Free up storage space
E003	Language model missing	Download language pack
E004	Sync failed	Try again when connected

A Voice Command Reference

A.1 Luganda Commands

Table 15 shows commands in Luganda.

Table 15: Luganda Voice Commands

Command	Translation	Example
"Naguze [item] [qty], credit ya [name]"	Give [qty] of [item] to [name] on credit	"Naguze matooke tatu, credit ya John"
"[name] yasasula [item]"	[name] paid for [item]	"John yasasula matooke"
"Mbulira ebya leero"	Tell me today's summary	"Mbulira ebya leero"

A.2 Swahili Commands

Table 16 shows commands in Swahili.

Table 16: Swahili Voice Commands

Command	Translation	Example
"Mpe [name] [qty] ya [item] kwa deni"	Give [qty] of [item] to [name] on credit	"Mpe John tatu ya matooke kwa deni"
"[name] alilipa [item]"	[name] paid for [item]	"John alilipa matooke"
"Muhtasari wa leo"	Today's summary	"Muhtasari wa leo"

B Offline Model Specifications

Table 17 shows offline model specifications.

Table 17: Offline Model Specifications

Language	Model Size	Accuracy	Memory Usage
Luganda	42MB	87%	~200MB RAM
Swahili	38MB	85%	~180MB RAM
English	40MB	90%	~200MB RAM

C Cost Estimates

C.1 Development Costs

Table 18 shows development cost estimates.

Table 18: Development Costs (Monthly)

Resource	Monthly Cost
Cloud Hosting	\$50-100
Speech Models	Free (Vosk)
Database Storage	\$20-50
Mobile App	\$0 (Native)
Total	\$70-150/month

C.2 Scalability Costs (10,000 Users)

Table 19 shows scalability cost estimates.

Table 19: Scalability Costs (10,000 Users)

Resource	Monthly Cost
Cloud Hosting	\$200-500
Sync Database	\$100-300
Backup Storage	\$50-100
Support	\$500-1000
Total	\$850-1900/month

Final Notes

This documentation is a living document. Please submit issues or pull requests for improvements.